

21. How does cross-validation help in hyperparameter tuning? What is grid search with cross-validation? What is random search with cross validation? How is cross-validation error calculated?

Role of Cross-Validation in Hyperparameter Tuning

Hyperparameters (like $n_estimators$ in Random Forests or C in SVMs) cannot be learned directly from the training data; they must be set before training begins. If we tune hyperparameters using only the training set, we select configurations that overfit the training data. If we use a single static test set, those hyperparameters leak information about the test set, ruining it as an unbiased evaluator. Cross-validation solves this by dynamically partitioning the data into multiple internal training and validation folds. This ensures that hyperparameter configurations are evaluated on unseen data, allowing us to find configurations that generalize well without contaminating the final testing pipeline.

Grid Search with Cross-Validation

Grid Search evaluates a manually pre-defined, discrete matrix of hyperparameter combinations exhaustively. For example, if you test 3 values for learning rate η and 3 values for regularization parameter C , Grid Search trains and evaluates the model across all 9 possible combinations (3×3). For each combination, it runs a full cross-validation routine and picks the parameters that yield the highest average validation score.

Random Search with Cross-Validation

Instead of checking every single point on a rigid grid, Random Search samples random parameter configurations from specified statistical distributions or bounded lists for a fixed number of iterations (n_iter).

- It is highly preferred over Grid Search when the hyperparameter search space is large or when some hyperparameters have a much greater impact on performance than others. It avoids wasting computation on minor changes to unimportant parameters.

Calculating Cross-Validation Error

For a chosen hyperparameter setting evaluated via K -fold cross-validation:

1. The dataset is split into K equal subsets.
2. The model is trained K times; each time, a unique fold k is held out as the validation set, and the remaining $K - 1$ folds serve as the training set.

3. For each of the K iterations, the performance error (such as Mean Squared Error, MSE_k) is calculated on the validation fold.
4. The final cross-validation error is the arithmetic mean of these K independent validation errors:

$$MSE_{CV} = \frac{1}{K} \sum_{k=1}^K MSE_k$$

22. What is K-fold cross-validation? Explain how K-fold cross-validation works step by step. What is stratified K-fold cross-validation?

K-Fold Cross-Validation Definition

K -fold cross-validation is a robust model evaluation protocol used to estimate a model's generalization performance on unseen data. It minimizes the statistical variance associated with a single train/test split by ensuring that every single data point gets an opportunity to be used exactly once for validation and $K - 1$ times for training.

Step-by-Step Methodology

- **Step 1: Partitioning:** Shuffle the entire dataset randomly and partition it into K equal-sized, mutually exclusive subsets (folds).
- **Step 2: Iteration Loop:** For each unique fold k (where $k = 1, 2, \dots, K$):
 - Select fold k to act as the temporary validation/testing set.
 - Combine the remaining $K - 1$ folds to form the temporary training set.
- **Step 3: Training:** Train the machine learning model on the temporary training dataset.
- **Step 4: Evaluation:** Evaluate the trained model on the held-out validation fold k and record its performance score (e.g., Accuracy or MSE).
- **Step 5: Aggregation:** Repeat steps 2 through 4 exactly K times, using a different fold for validation each time. Calculate the final performance estimate by averaging the K individual validation scores.

Stratified K-Fold Cross-Validation

Stratified K -fold is a specialized variation engineered specifically for classification tasks with severe class imbalances. During partitioning, it ensures that **each fold contains approximately the same percentage of each target class label as the complete original dataset**. This structural constraint prevents scenarios where a fold accidentally contains zero positive samples, which would yield destabilized, highly skewed error metrics.

23. What is cross-validation? Why do we use cross-validation in machine learning? What is the goal of model validation? What is the difference between training, validation, and test sets?

Cross-Validation & The Goal of Model Validation

Cross-validation is a systematic resampling framework used to assess how reliably a statistical model generalizes to an independent, completely unseen dataset. The core goal of model validation is to detect and mitigate underfitting and overfitting before deploying a model. It helps us confirm that the model has learned the actual underlying physical and statistical relationships rather than simply memorizing the training data's localized noise.

Structural Differences Between Data Splits

To maintain strict validation boundaries, data is split into three distinct sets:

Dataset Split	Role / Objective
Training Set	The primary data pool used by the learning algorithm to directly update the model's internal parameters (such as weights and biases).
Validation Set	An intermediate data pool used to evaluate the model's performance while tuning hyperparameters or selecting architectures. It acts as an unbiased proxy for unseen data to guide hyperparameter choices.
Test Set	A completely isolated data pool used <i>only</i> after training and tuning are completely finished. It provides a final, completely unbiased evaluation of the model's real-world generalization performance.

24. How does a decision tree make predictions? What is a root node, decision node, and leaf node? What is splitting in a decision tree? What is the stopping criterion in tree construction?

Predictive Mechanism

A Decision Tree makes predictions by routing input instances down a hierarchical chain of conditional, binary rules (if/then questions) applied to its feature values. Starting at the very top, the instance traverses down specific branches based on whether it satisfies each threshold condition until it lands in a terminal base.

Node Hierarchy Definitions

- **Root Node:** The topmost node of the tree architecture. It represents the entire initial dataset before any filtering or partitioning takes place.
- **Decision Node:** An intermediate sub-node that contains a conditional feature split threshold. It breaks data down into two or more distinct sub-pathways.
- **Leaf Node:** A terminal node at the bottom of a branch that contains no further downstream splits. It outputs the final prediction—either a majority class label for classification or an average target value for regression.

Splitting and Stopping Criteria

- **Splitting:** The recursive process of dividing a parent node's dataset into two or more child subsets. The algorithm evaluates all available features and thresholds, choosing the specific split that maximizes data purity (homogeneity) within the resulting child nodes.
- **Stopping Criteria:** Constraints that stop the tree from expanding infinitely, preventing it from growing down to a single sample per leaf (which causes severe overfitting). Common stopping criteria include:
 - Reaching a predefined maximum tree depth (`max_depth`).
 - The number of samples in a node falls below a threshold (`min_samples_split`).
 - The purity improvement from a split falls below a minimum threshold.

25. What is gini impurity in decision trees?

Definition and Mathematical Formula

Gini Impurity is a metric used by classification decision tree algorithms (such as CART) to evaluate the quality and purity of a proposed split. It measures the probability that a randomly chosen element from a specific node's dataset would be incorrectly labeled if it were classified completely at random based on the overall distribution of labels in that node.

The Gini Impurity (p) for a node containing C distinct classes is calculated as:

$$(p) = 1 - \sum_{i=1}^C p_i^2$$

Where p_i is the probability or relative frequency of an instance belonging to class i within that specific node.

Practical Interpretation

- **= 0 (Perfect Purity):** Occurs when all samples inside a node belong to a single class. The node is perfectly homogeneous, making it a natural leaf node.

- **Maximum Impurity:** Occurs when classes are distributed perfectly evenly across a node (e.g., a 50/50 split in a binary classification problem, yielding $= 0.5$).
- When constructing trees, the algorithm calculates the Gini Impurity of a parent node and subtracts the weighted Gini Impurities of the proposed child nodes. It selects the split that yields the largest drop in impurity, a property known as **Information Gain**.

26. How does pruning improve generalization?

Decision trees are highly non-parametric and flexible; if left unchecked, they will continue splitting until they perfectly memorize every training example, including random noise and outliers. This leads to low training error but high validation error (**severe overfitting**).

Pruning improves a model's generalization performance by cutting back non-essential, deeply nested branches that capture localized noise rather than broad statistical signals.

Mechanics of Generalization Improvement

- **Simplifies the Decision Space:** Pruning collapses complex, highly specific sub-branches back into broader leaf nodes, smoothing out jagged decision boundaries.
- **Reduces Model Variance:** By removing fragile splits that depend heavily on a few training samples, pruning makes the tree structure far more stable when evaluated on new data.
- **Balances the Tradeoff:** It trades a minor increase in training bias for a substantial reduction in model variance, minimizing total expected generalization error.

27. How do you decide optimal tree depth? Why are decision trees prone to overfitting?

Deciding Optimal Tree Depth

The optimal tree depth is a critical hyperparameter that must be tuned carefully using validation protocols:

1. **Cross-Validation Tuning:** Train multiple trees across a range of maximum depths (`max_depth`) and track their performance using cross-validation.
2. **Elbow/Loss Curve Analysis:** Plot the training error and validation error together as a function of tree depth.
 - As depth increases, training error drops monotonically. Validation error, however, will typically decrease to an optimal point and then begin climbing. The depth corresponding to the lowest validation error marks the optimal trade-off point.

Why Decision Trees are Highly Prone to Overfitting

1. **Unconstrained Learning Flexibility:** Decision trees do not assume any rigid parametric shape (unlike linear models). They can create infinitely complex boundaries to accommodate every data point if left unconstrained.
2. **Greedy Splitting Stratagem:** The algorithm splits data using a recursive, greedy approach that optimizes purity locally at each step. It does not check whether a sequence of splits will overfit downstream, making it highly susceptible to localized anomalies and noise in the training set.

28. Random forest: What is the effect of increasing number of trees? What is bootstrap parameter? How does max features affect performance?

Increasing the Number of Trees (`n_estimators`)

A Random Forest is an ensemble method that averages the predictions of many independent decision trees to reduce variance.

- **Effect:** Adding more trees improves the stability of the ensemble and reduces variance without increasing bias. Unlike standard decision trees, **increasing the number of trees in a Random Forest does not cause overfitting**. Once you train enough trees to stabilize the averaged voting error, adding more trees simply hits a plateau of diminishing returns while consuming more computational time.

The Bootstrap Parameter

The bootstrap parameter dictates whether individual trees are trained on resampled subsets drawn with replacement from the master training dataset.

- **Effect:** Enabling bootstrapping (`bootstrap=True`) ensures that each tree sees a slightly different variation of the data, with roughly 3.2% of the original samples present and the rest left out as Out-Of-Bag (OOB) data. This injection of data diversity decorrelates the trees, which is essential for reducing the ensemble's overall variance.

The Maximum Features Parameter (`max_features`)

This hyperparameter restricts the number of random features evaluated as split candidates at each node.

- **Small `max_features` :** Strongly forces feature diversity across trees, which decorrelates their predictions and reduces ensemble variance. However, setting it too low can limit a tree's ability to find strong predictive splits, increasing bias.
- **Large `max_features` :** Trees can choose from a wider selection of features, which makes individual trees stronger but also more similar to one another. This increases the correlation

between trees, reducing the variance-reduction benefits of the ensemble.

29. What is Chi-Square pruning in decision trees?

Definition and Core Mechanics

Chi-Square (χ^2) Pruning is a statistical, bottom-up pruning technique used primarily during the construction of classification decision trees to determine whether a proposed split actually provides a statistically significant improvement.

It tests the null hypothesis (H_0) that a proposed split's distribution of target classes occurs entirely by random chance, meaning the feature split has no true relationship with the target variable.

Step-by-Step Execution

1. For a given node, calculate the **Expected Frequency** (E_{ij}) of each class in the resulting child nodes if the split were purely random.
2. Count the **Observed Frequency** (o_{ij}) of classes that actually fall into those child nodes after applying the split condition.
3. Compute the standard Chi-Square test statistic:

$$\chi^2 = \sum \frac{(o_{ij} - E_{ij})^2}{E_{ij}}$$

4. Convert this statistic into a p -value based on the split's degrees of freedom.
5. Compare the p -value against a selected significance threshold (e.g., $\alpha = 0.05$):
 - If $p \leq \alpha$, the split is statistically significant and is kept.
 - If $p > \alpha$, the null hypothesis cannot be rejected. This means the split's apparent purity improvement is likely just a random statistical anomaly, so the split is pruned away to prevent overfitting.

30. What is Lasso/Ridge Regression? How is Lasso different from Ridge Regression? Why is Lasso called feature selection method? Write Ridge or Lasso cost function.

Core Concepts

Lasso and Ridge are regularized extensions of Ordinary Least Squares (OLS) linear regression. They mitigate overfitting and address multicollinearity by appending a penalty term directly to the standard sum of squared errors cost function, discouraging model coefficients from growing excessively large.

Cost Functions

- **Ridge Regression Cost Function (L_2 Regularization):**

$$J(\beta) = \sum_{i=1}^m (y_i - \mathbf{x}_i^T \beta)^2 + \alpha \sum_{j=1}^n \beta_j^2$$

- **Lasso Regression Cost Function (L_1 Regularization):**

$$J(\beta) = \sum_{i=1}^m (y_i - \mathbf{x}_i^T \beta)^2 + \alpha \sum_{j=1}^n |\beta_j|$$

Where $\alpha \geq 0$ is the regularization hyperparameter controlling the severity of the penalty.

Structural Differences

Feature	Ridge Regression (L2)	Lasso Regression (L1)
Penalty Type	Penalty is proportional to the sum of squared weights (β_j^2).	Penalty is proportional to the sum of absolute weights ($ \beta_j $)
Weight Shrinkage	Shrinks coefficients smoothly asymptotically toward zero, but never forces them to exactly zero.	Drives less important coefficients to exactly zero .
Geometry	The penalty boundary is a smooth hypersphere, leading to solutions where weights remain small but non-zero.	The penalty boundary has sharp corners (a hyperdiamond), which frequently clips coefficients exactly at zero.

Why Lasso acts as a Feature Selection Method

Because Lasso's L_1 absolute penalty drives uninformative or redundant feature weights down to exactly zero, it completely removes those variables from the final model equation. This leaves a sparse model containing only the most predictive features, effectively performing **automated embedded feature selection**. Ridge, by contrast, retains all features with small, non-zero weights.